

Shell

- a program that accepts and interprets commands
- provides an interface to the system
- the shell is configured for each individual user

There are many shells

- **bash** - the GNU Bourne Again Shell is the most common
- **sh** - bash goes by the name of sh, it's often a pointer to the bash shell

Two types of default shells:

1. **Default interactive shell** (default)
used by the user to enter commands
2. **System shell**
used by Linux system to run shell scripts, usually at startup
the file `/bin/sh` is a pointer to the default system shell:
 - normally `/bin/bash` for linux
 - Ubuntu is `/bin/dash`

Using a shell

Most commands are external - they're programs external to the shell

A few commands are internal

Internal commands take precedence

Command Tricks

Command Completion

type the start of a command/filename and press `tab`
the shell tries to fill in the rest of the command or filename

history

shows previously used commands

`history -c` clears the history

the bash history is stored in `.bash_history` file

Previous command

`↑ ↓` - move up/down in history

Search for a command

`ctrl+R` search back in history

`ctrl+S` search forward in history

begin typing characters

characters you type don't need to be at the beginning of the command, they can exist anywhere in the command

Moving within a line

`← →`

`ctrl+→` move one word at a time

`ctrl+A` start of line

`ctrl+E` end of line

Deleting Text

`ctrl+K` deletes from cursor to end of line

`ctrl+X` deletes from cursor to beginning of line

Change case`esc+U` cursor to end of word to uppercase`esc+L` lowercase

Internal Commands

Built-in commands

cd

change directory

`cd /home/james``~` stands for the users home directory**pwd**

print working directory

echo

prints the text you enter

good way to see contents of variables

`echo $PATH`**time**

how long command takes to execute

`time pwd` tells you how long it too to execute the `pwd` command

three times are displayed:

1. total execute time
2. user CPU time
3. system CPU time

set

displays all shell variables/functions

including environment variables and shell internal variables

you can enable/disable options:

`set -x` turns on debugging`set -- arg1 arg2` sets positional parameters \$1, \$2 to arg1 and arg2**exit**

terminates the shell

logoutterminates only *login shells**login shells* automatically launch when you log in
a terminal on a GUI is not a login shell ✗

SSH into remote server ✓

type

checks if a command is a built-in

```
james@laptop:~$ type cd #example1
cd is a shell builtin
james@laptop:~$ type bash #example2
bash is /usr/bin/bash
```

example1 `cd` is in-built command

example2 `usr/bin/bash` is the path to the executable file - it's a standalone program

Some commands are **uplicated** - has an internal and external executable/s
internal commands take precedence

to check use `-a`

```
james@laptop:~$ type -a cd
cd is a shell builtin
james@laptop:~$ type -a pwd
pwd is a shell builtin
pwd is /usr/bin/pwd
pwd is /bin/pwd
```

Getting help

man

manual

summary of a command/file

```
man pwd
```

Less pager

`man` uses the less pager to display info
displays text one page at a time

<code>spacebar</code>	move forward one page
-----------------------	-----------------------

arrow keys	move up/down one line
------------	-----------------------

<code>/</code>	Search for text
----------------	-----------------

<code>q</code>	quit
----------------	------

info

same as man but uses a different text format

```
info pwd
```

help

for **builtin** commands

```
help pwd
```

More Commands

mkdir

create directory

```
mkdir test
```

touch

create file

`touch one.txt`**ls**

list files and folders in directory

cat

display file contents to STDOUT

`cat file.txt`**PATH**

- When you type a command that's not recognised as an internal command, it checks the path to find a program by that name to execute it.
- The *path* is a **colon-delimited** list of directories in which commands can be found
- Defined by `$PATH` environment variable

Running programs

- You can run programs that aren't on the path by using the path name
- `./someprog` runs someprog in the current directory
- `home/user/someprog` runs it in the *home/user* directory

To run a program it must be marked as executable

Standard programs are marked as executable